

Applied Deep Learning

Chapter 6: Recurrent NNs

Ali Bereyhi

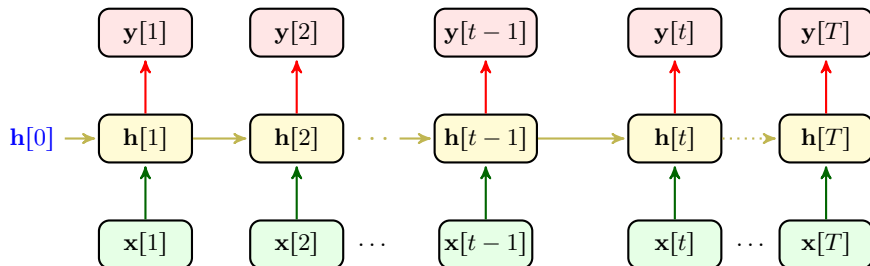
`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Training our Shallow RNN

We now want to train this basic RNN



Let's consider training for only one sample

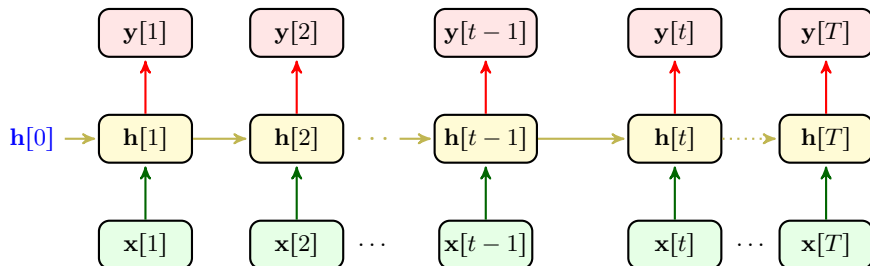
We assume that we have a sequence of labels $v[1], \dots, v[T]$

- ① We know *some* $v[t]$ *could be empty*, e.g., in many-to-one scenario
- ② So *could be some* of $x[t]$'s, e.g., in one-to-many scenario

We however have no problem with that!

Training our Shallow RNN

We now want to train this basic RNN



Let's consider training for only one sample

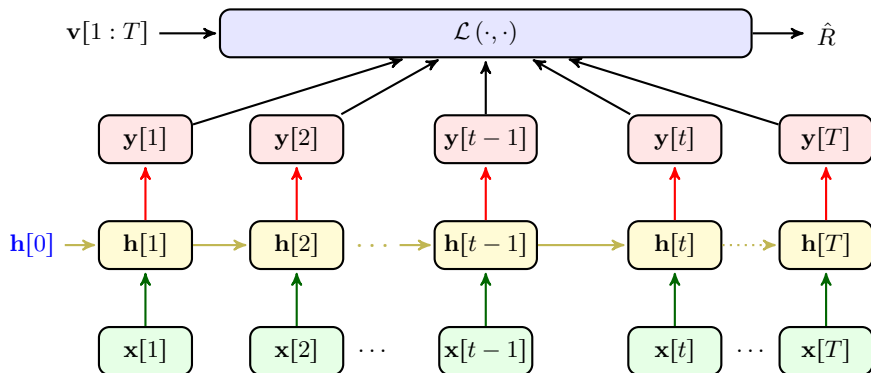
The loss in general can be written as

$$\hat{R} = \mathcal{L}(\mathbf{y}[1:T], \mathbf{v}[1:T])$$

where we use shorten notation $\mathbf{y}[1:T] = \mathbf{y}[1], \dots, \mathbf{y}[T]$

Training our Shallow RNN

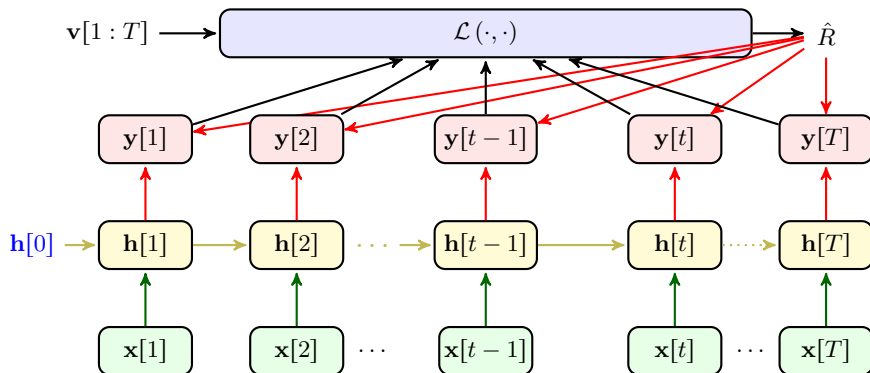
We can think of such a diagram



- + It seems to be hard to get back from $\hat{R}$ to each $y[t]$
- Well! That's true, but we have some remedy for it

Training our Shallow RNN

We can think of such a diagram



For the moment, we assume that we can

compute the $\nabla_{\mathbf{y}[t]} \hat{R}$ for all $t \equiv$ *move backward from $\hat{R}$ to any $\mathbf{y}[t]$*

Backward Pass Through Time

Starting from $\hat{R}$, say we want to find $\nabla_{\mathbf{w}_m} \hat{R}$

- Since $\hat{R}$ is a function of $\mathbf{y}[1 : T]$, we should write a *vectorized chain rule*

$$\begin{aligned}\nabla_{\mathbf{w}_m} \hat{R} &= \nabla_{\mathbf{y}[1]} \hat{R} \circ \nabla_{\mathbf{w}_m} \mathbf{y}[1] + \dots + \nabla_{\mathbf{y}[T]} \hat{R} \circ \nabla_{\mathbf{w}_m} \mathbf{y}[T] \\ &= \sum_{t=1}^T \nabla_{\mathbf{y}[t]} \hat{R} \circ \nabla_{\mathbf{w}_m} \mathbf{y}[t]\end{aligned}$$

- Since we assumed that we have $\nabla_{\mathbf{y}[t]} \hat{R}$, our main task is to find

$$\nabla_{\mathbf{w}_m} \mathbf{y}[t]$$

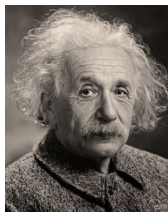
for all t : we could hence compute it for a *general t*

↳ This is a tensor-like gradient, i.e., $[\nabla_{\mathbf{w}_m} y_1[t], \dots, \nabla_{\mathbf{w}_m} y_M[t]]$

- Apparently, we should apply chain rule for several times!

Backward Pass Through Time

- + *But, this is going to be exhausting?*
- Well, we could again follow Albert Einstein advice!



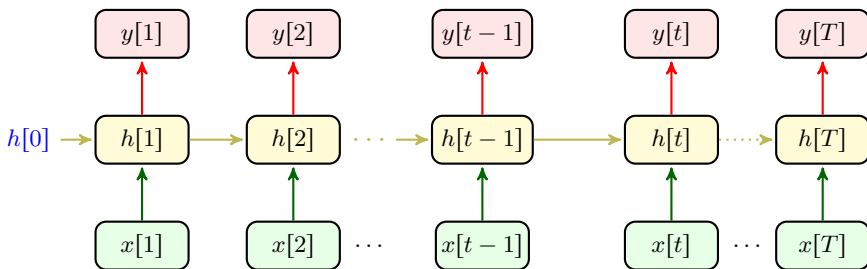
*"Everything should be made as **simple** as possible, but not **simpler**!"*

Backward Pass Through Time: *Simple Example*

Let's consider a dummy RNN with all variables being *scalar*

- 1 We have $y[t] = f(w_2 h[t])$
- 2 We have $h[t] = f(w_1 x[t] + w_m h[t-1])$
- 3 We start with *hidden state* $h[0]$

So the diagram gets simplified as below



Backward Pass Through Time: *Simple Example*

Starting from $\hat{R}$, say we want to find $\nabla_{w_m} \hat{R}$: *our main task is to find*

$$\frac{\partial y[t]}{\partial w_m}$$

Let's start the computation: say we have passed forward through the RNN

- *we now have $y[t]$, $h[t]$ and $x[t]$ for all t*

To go backward, we note that $y[t] = f(w_2 h[t])$

- *$y[t]$ is a function of $h[t]$: so we write the **chain rule** as*

$$\frac{\partial y[t]}{\partial w_m} = \frac{\partial y[t]}{\partial h[t]} \frac{\partial h[t]}{\partial w_m}$$

↳ *we can compute the first term as $\partial y[t] / \partial h[t] = w_2 \dot{f}(w_2 h[t])$*

↳ *we should compute the second term by further chain rule*

Backward Pass Through Time: *Simple Example*

$$\frac{\partial y[t]}{\partial w_m} = w_2 \dot{f}(w_2 h[t]) \frac{\partial h[t]}{\partial w_m}$$

We keep going backward, by noting that $h[t] = f(w_1 x[t] + w_m h[t-1])$

- $h[t]$ is a function of w_m and $h[t-1]$:¹

↳ let's write $h[t] = g(w_m, h[t-1])$

- So we write the *chain rule* as

$$\begin{aligned} \frac{\partial h[t]}{\partial w_m} &= \frac{\partial g}{\partial w_m} \underbrace{\frac{\partial w_m}{\partial w_m}}_1 + \frac{\partial g}{\partial h[t-1]} \frac{\partial h[t-1]}{\partial w_m} \\ &= \frac{\partial g}{\partial w_m} + \frac{\partial g}{\partial h[t-1]} \frac{\partial h[t-1]}{\partial w_m} \end{aligned}$$

¹We ignore w_1 and $x[t]$, as they are obviously not functions of w_m

Backward Pass Through Time: *Simple Example*

$$\frac{\partial y[t]}{\partial w_m} = w_2 \dot{f}(w_2 h[t]) \frac{\partial h[t]}{\partial w_m}$$

We keep going backward, by noting that $h[t] = f(z[t])$

- Let's define $z[t] = w_1 x[t] + w_m h[t-1]$
- $h[t] = g(w_m, h[t-1])$
 - ↳ we can compute $\partial g / \partial w_m = h[t-1] \dot{f}(z[t])$
 - ↳ we can compute $\partial g / \partial h[t-1] = w_m \dot{f}(z[t])$
- So we can simplify the *chain rule* as

$$\begin{aligned} \frac{\partial h[t]}{\partial w_m} &= h[t-1] \dot{f}(z[t]) + w_m \dot{f}(z[t]) \frac{\partial h[t-1]}{\partial w_m} \\ &= \dot{f}(z[t]) \left(h[t-1] + w_m \frac{\partial h[t-1]}{\partial w_m} \right) \end{aligned}$$

Backward Pass Through Time: *Simple Example*

$$\frac{\partial y[t]}{\partial w_m} = w_2 \dot{f}(w_2 h[t]) \dot{f}(z[t]) \left(h[t-1] + w_m \frac{\partial h[t-1]}{\partial w_m} \right)$$

We keep going backward: $h[t-1] = f(z[t-1])$

↳ Recall that $z[t-1] = w_1 x[t-1] + w_m h[t-2]$

- We just need to replace t with $t-1$ in the last derivation

$$\frac{\partial h[t-1]}{\partial w_m} = \dot{f}(z[t-1]) \left(h[t-2] + w_m \frac{\partial h[t-2]}{\partial w_m} \right)$$

Backward Pass Through Time: *Simple Example*

Backpropagation at time t is hence described by a pair of recursive equations

- At time t , we compute

$$\frac{\partial y[t]}{\partial w_m} = w_2 \dot{f}(w_2 h[t]) \frac{\partial h[t]}{\partial w_m}$$

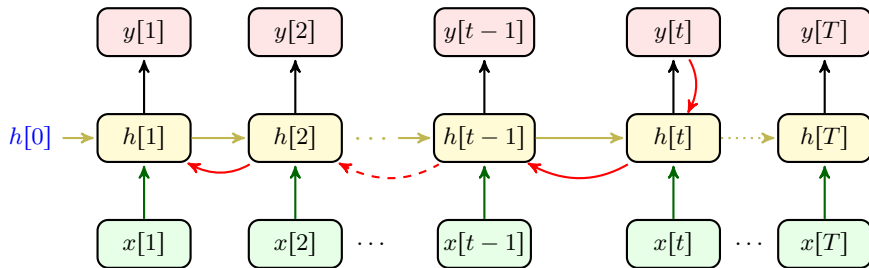
- For each $i = t, t - 1, \dots, 1$, we use

$$\frac{\partial h[i]}{\partial w_m} = \dot{f}(z[i]) \left(h[i - 1] + w_m \frac{\partial h[i - 1]}{\partial w_m} \right)$$

- We stop at $i = 1$, where we get

$$\frac{\partial h[1]}{\partial w_m} = \dot{f}(z[1]) \left(h[0] + \underbrace{w_m \frac{\partial h[0]}{\partial w_m}}_0 \right) = \dot{f}(z[1]) h[0]$$

Backward Pass Through Time: *Simple Example*



Key Point

Propagating *back in time* is described via a *recursive equation*

Backward Pass Through Time: *Simple Example*

Recursive equation has an interesting feature

- Say we have backpropagated from time t

$$\frac{\partial y[t]}{\partial w_m} = w_2 \dot{f}(w_2 h[t]) \frac{\partial h[t]}{\partial w_m}$$

↳ We hence have $\partial h[t]/\partial w_m, \partial h[t-1]/\partial w_m, \dots, \partial h[1]/\partial w_m$

- Now, if we want to backpropagate from $t-1$

↳ We *already have* $\partial h[t-1]/\partial w_m, \partial h[t-2]/\partial w_m, \dots, \partial h[1]/\partial w_m$

↳ We do *not* need to backpropagate through time anymore

Moral of Story

Pass *forward* till *end of sequence* and *backpropagate* to the beginning *just once*

Backpropagation Through Time (BPTT)

BPTT() :

- ① Start at t with $\nabla_{\mathbf{W}_m} \mathbf{y}[t] = \mathbf{W}_2 \circ \dot{f}(\mathbf{W}_2 \mathbf{h}[t]) \circ \nabla_{\mathbf{W}_m} \mathbf{h}[t]$
- ② Go back in time as $\nabla_{\mathbf{W}_m} \mathbf{h}[i] = \dot{f}(\mathbf{z}[i]) \circ (\mathbf{h}[i-1] + \mathbf{W}_m \circ \nabla_{\mathbf{W}_m} \mathbf{h}[i-1])$
- ③ Stop at $i = 1$ with $\nabla_{\mathbf{W}_m} \mathbf{h}[1] = \dot{f}(\mathbf{z}[1]) \circ \mathbf{h}[0]$

- + It looks very similar to backpropagation in deep NNs!
- Exactly! Even simple RNN is very *deep* through time
- + Don't we experience *vanishing* or *exploding* gradient then!
- Yes! Let's check it out

BPTT: Vanishing Gradient

Let's expand the gradient in our example

$$\begin{aligned}
 \frac{\partial y[t]}{\partial w_m} = & \underbrace{w_2 \dot{f}(w_2 h[t]) \dot{f}(z[t])}_{\alpha_1} h[t-1] \\
 & + \underbrace{w_2 \dot{f}(w_2 h[t]) w_m \dot{f}(z[t]) \dot{f}(z[t-1])}_{\alpha_2} h[t-2] + \dots \\
 & + \underbrace{w_2 \dot{f}(w_2 h[t]) w_m^{i-1} \left(\prod_{j=0}^{i-1} \dot{f}(z[t-j]) \right)}_{\alpha_i} h[t-i] + \dots \\
 & + \underbrace{w_2 \dot{f}(w_2 h[t]) w_m^{t-1} \left(\prod_{j=0}^{t-1} \dot{f}(z[t-j]) \right)}_{\alpha_t} h[0]
 \end{aligned}$$

BPTT: Vanishing Gradient

$$\frac{\partial y[t]}{\partial w_m} = \sum_{i=1}^t \alpha_i h[t-i]$$

This says that gradient of hidden state in i steps back in time is multiplied by

$$\alpha_i = w_2 w_m^{i-1} \dot{f}(w_2 h[t]) \left(\prod_{j=0}^{i-1} \dot{f}(z[t-j]) \right)$$

Recall **deep** NNs: we could see two cases

- If $\dot{f}(\cdot) > 1$ most of the time
 - ↳ **very old** hidden states can **explode** the gradient
- If $\dot{f}(\cdot) < 1$ most of the time
 - ↳ **very old** hidden states **have pretty much no impact on the gradient**
 - ↳ this **typically** happens and we call it **vanishing gradient through time**

Handling Vanishing Gradient Through Time

- + Cant we use the same remedy as in deep NNs?
- Yes and No!

It is important to note that *vanishing gradient through time* is a bit *different*: we are still updating weights with *large enough gradients*, but these gradients have *no information* of *long-term memory*

Solutions to *vanishing gradient through time* are mainly

- Use $\tanh(\cdot)$ *activation*
 - ↳ $\tanh(\cdot)$ is able to keep memory for a longer period
- Use *truncated BPTT*
 - ↳ Repeat short-term BPTTs every couple of time intervals
- Invoke *gating approach*
 - ↳ This is the most *sophisticated approach*
 - ↳ It was known for a long time, but received attention much later!